Winning* the 2025 FIT5222 Pacman Capture the Flag Challenge*

Sean Murphy
smur0055@student.monash.edu

Abstract

This report details the design, implementation, and evaluation of an AI controller for the Pacman Capture the Flag (CTF) challenge, a complex, multi-agent, and partially-observable planning domain. To address the problem's core challenges, two foundational components were developed: a probabilistic Bayesian filter for robust opponent tracking and a topological map preprocessor for strategic state abstraction.

Two primary agent architectures were implemented and compared. The first, ***CoopPDDL****, fulfilled the assignment's requirements by implementing a decoupled planning system. This agent featured a high-level PDDL planner enhanced with cooperative predicates and a low-level approximate Q-learning planner with specialized, task-specific reward functions. The second,* ***AdvantageMax****, is a novel, model-based agent that forgoes formal planning. Instead, it introduces a powerful differential heuristic, termed "Advantage," which quantifies territorial control by calculating the difference between ally and opponent costs to reach any key location. This agent uses a greedy, one-step lookahead search to maximize this heuristic value.*

Experimental evaluation from the competition server shows that ***AdvantageMax*** *achieved a first-place ELO ranking, demonstrating superior performance by a significant margin. A critical discussion is also provided, analyzing the failure of several end-to-end Deep Reinforcement Learning (DRL) approaches (using GNNs, CNNs, MCTS, and Population-Based Training) which were unable to overcome the domain's sparse rewards and state representation challenges. The report concludes that for this real-time, adversarial domain, a well-engineered, domain-specific heuristic was demonstrably more effective than both the formal PDDL planning architecture and the complex DRL-based methods.*

*Ok, so I haven't won any tournament YET, but this title is a bold prediction given that the Cherry Cup results will only be released after this report's due date. At the time of writing, my agent is trading 1st and 2nd place on the ELO leaderboard, which I'm choosing to interpret as "basically already won." Future me will either look prophetic or hilariously overconfident—no middle ground.

1. Introduction

The Pacman Capture the Flag (CTF) challenge is a robust benchmark for multi-agent planning, originally developed as part of the UC Berkeley introductory AI course projects by DeNero and Klein [1]. The task requires the design of an AI controller for a team of two agents competing against an opposing team in a variation of the classic Pacman game. The primary objective is to capture food dots from the opponent's territory and return to the home territory to score points, all while defending one's own food.

The Pacman CTF environment is a complex, multi-faceted planning domain, representing a classic *multi-agent pathfinding (MAPF) problem* under adversarial conditions. It is characterized as:

- **Multi-agent:** Agents must cooperate with a teammate while competing against two opponents.
- **Dynamic:** The environment state changes as food is consumed.
- **Discrete & Sequential:** The game progresses in discrete time steps, and past decisions directly affect future states.
- **Offline & Deterministic:** The simulation pauses to allow each agent to deliberate, and the outcome of an action is deterministic.
- **Partially Observable:** Agents have a limited sensing range of 5 squares (Manhattan distance) and only receive noisy distance readings for unobserved opponents. This transforms the otherwise deterministic environment into a *partially observable planning problem (POMDP)*, where agents must act based on an uncertain belief state rather than perfect information.

A game ends when one team eats all but two of the opponent's dots, or when the 1200-action time limit (300 moves per agent) is reached. The central engineering challenge is to design an agent controller that can reason effectively under these rules. The assignment mandates a framework that separates **high-level strategy** (deciding what goals to pursue) from **low-level strategy** (generating the concrete actions to achieve them). This mandated architecture is a form of *decoupled planning*, where high-level strategic goals are determined first and then passed to a pathfinding controller for execution. This high-level reasoning is a strict requirement and must incorporate the *PDDL planning language*.

This report details the design, implementation, and evaluation of an agent controller developed to solve this specific challenge. It describes the evolution from a PDDL-centric agent, which achieved a top-10 leaderboard rank, to a final, more robust low-level-optimised agent, achieving a top-1 tournament finish. The architectures of both major approaches are discussed, followed by an experimental analysis justifying the final design. The report also reflects on several alternative, ultimately unsuccessful, approaches, before concluding with the key lessons learned from the competition.

2. Approaches

The development of a successful agent required moving beyond the provided baseline. This section begins by describing two architectural innovations that were developed to support both implementations, followed by details on the two agent implementations: **CoopPDDL** and **AdvantageMax**. We discuss how each approach improves upon the baseline and how they differ, and then conclude with a comparison of the two agents.

2.1. Key Innovations

2.1.1. Probabilistic Opponent Tracking

The first innovation was a probabilistic opponent tracking module. As the environment is a *partially observable planning problem (POMDP)*, a simple reactive agent will fail. To act effectively, an agent must maintain a belief over unobserved states. A probabilistic opponent tracking module was developed to manage this **belief state**. This involves maintaining a full 2D belief distribution—a grid of probabilities—for each of the two opponents. This belief grid is updated at every time step for the acting agent, providing a persistent and more accurate estimate of enemy locations.

The update process follows a *standard Bayesian filtering model*, which is a practical and effective approach for state estimation in POMDPs. Algorithm 1 shows the pseudocode for this entire process.

The key steps of this algorithm are:

1. **Exact Observation:** If the opponent is visible (within 5 squares of any ally), the belief distribution collapses to a 100% certainty at that exact position.
2. **Time Update (Propagation):** If the opponent is not visible but did just move, the algorithm performs a time update. We propagate beliefs by taking the prior belief and "smearing" the probability from each cell to its legal, non-wall neighbors, simulating one step of movement.
3. **Observation Update (Correction):** The algorithm then incorporates the noisy distance reading using a Bayesian update. For every possible position (x, y), it calculates:

$$P(\text{pos}|\text{obs}) \propto P(\text{obs}|\text{pos}) \times P(\text{pos})$$

Algorithm 1 Probabilistic Belief Update for a Single Opponent

```
function UpdateBeliefs(prevBelief, opponent)
    exactPos ← exact position of opponent if visible
    if exactPos is not null then
        return exactPos
    end if
    priorBelief ← UpdatePrior(prevBelief, opponent)
    postBelief ← UpdatePost(priorBelief, opponent)
    return normalized postBelief
end function

function UpdatePrior(belief)
    newBelief ← empty grid
    for each position pos where belief[pos] > 0 do
        prob ← belief[pos]
        legalNeighbors ← legal neighboring positions of pos
        probPerNeighbor ← prob ÷ |legalNeighbors|
        for each neighbor in legalNeighbors do
            newBelief[neighbor] ← newBelief[neighbor] + probPerNeighbor
        end for
    end for
    return newBelief
end function

function UpdatePost(priorBelief, opponent)
    noisyDist ← noisy distance observation to opponent
    myPos ← agent's current position
    postBelief ← empty grid
    for each position pos where priorBelief[pos] > 0 do
        actualDist ← maze distance from pos to myPos
        likelihood ← probability of observing noisyDist given actualDist
        postBelief[pos] ← priorBelief[pos] × likelihood
    end for
    return postBelief
end function
```

Where $P(\text{pos})$ is the prior belief from the time update, and $P(\text{obs}|\text{pos})$ is the likelihood of receiving that noisy distance reading given the opponent was at (x, y).

4. **Normalization:** Finally, the posterior belief grid is normalized so all probabilities sum to 1.

This entire process is called by each agent when it's their turn to move to update its beliefs about both opponents. This module provides a significantly richer representation of the environment. The resulting probability grid can be used to find the most likely position of an opponent, calculate the probability of an opponent being in a specific region, or compute an "expected risk" for pathfinding. This belief system serves as a foundational component for providing richer information for both implementations.

2.1.2. Strategic Map Preprocessing

During early testing for this project, I realized that for any given agent, there were only a handful of positions on the map where they needed to make a decision. Only positions that had three or more neighbours require the agent to make a decision of which direction to move. For positions with

fewer than three neighbours, the agent can only continue moving forward or backtracking on their previous steps.

This discovery allowed for a powerful **spatial abstraction**, reframing the raw *grid-based domain representation* as a more efficient topological graph. This new representation features:

- **Junctions:** These are the "decision points" in the maze, typically locations with three or more legal moves. They are identified first and serve as the nodes in the strategic graph.
- **Corridors:** These are the simple, non-branching paths that connect two junctions. They represent the graph's edges and allow the agent to reason about "lanes" or "hallways" as single entities.
- **Articulation Points:** Also known as cut vertices, these are critical junctions or tiles that, if blocked, would split a section of the map into two disconnected components. These are the map's "chokepoints." Identifying these is analogous to *landmark detection* in classical planning, as any valid path to a 'dead-end zone' must pass through these critical tiles.
- **Dead-End Zones:** These are entire regions of the map (often containing food) that are only accessible via a single articulation point.

A visualisation of these different features is shown in Figure 1.

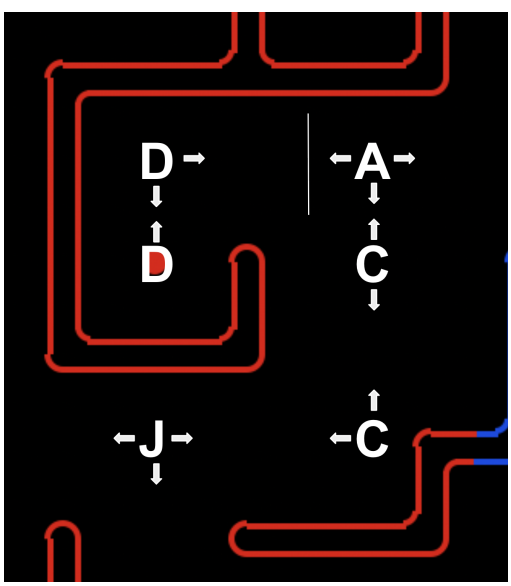

Figure 1. Topological map with labelled features including: (J)unctions, (C)orridors, (A)rticulation points, and (D)ead-end zones.

This preprocessed strategic map served as a powerful tool. By reasoning over this *abstracted state space*, the planner can make decisions at a much higher level, avoiding the *combinatorial explosion* of planning at the raw grid level. For example, a high-level "chase enemy" goal can be set if we sense an enemy is in a dead-end zone and we can get to the articulation point before they escape.

2.2. Implementation 1: CoopPDDL

CoopPDDL is a direct extension of the baseline `staffTeam.py` agent. It retains the same architecture as the baseline. At the high-level, this involves *formulating the current game state as a PDDL problem*, determining a goal based on these predicates, and querying a PDDL-solver for a list of high-level actions. At the low-level, we define a set of relevant features for each high-level action and use Q-learning to learn a set of weights for each feature, allowing us to determine the best next action.

2.2.1. High-Level Planner

The high-level planner follows the same structure as the baseline and can be split up into three parts:

1. Extracting the PDDL state from the environment.
2. Determining a goal state based on the extracted state.
3. Using a PDDL solver to find a list of predefined high-level actions.

At each of these steps, this implementation significantly improves on the baseline.

PDDL State Extraction This implementation improves on the baseline by including a richer set of cooperative and comparative features in the PDDL state. The most critical improvements are the introduction of predicates that facilitate teammate coordination.

My implementation uses shared class variables to track the team's intended high-level actions. For example the predicate `(ally_chasing, enemy_obj)` allows the agent to know whether our teammate is chasing a specific opponent. These cooperative predicates prevent both agents from "stepping on each other's toes" which was a common issue in the baseline. This mechanism acts as a form of **dynamic, prioritised planning**, allowing the agents to 'decouple' their goals and select complementary roles based on which agent is better positioned.

Furthermore, new predicates were introduced for tracking relative positioning between agents. For example, `(closer_to_enemy_obj)` allows the agent to know whether, compared to its own teammate, it is closer to a certain opponent, allowing for more reliable task allocation where agents closer to enemies can assume a defensive stance, whereas agents with more space can try to attack.

Ultimately, the ability to calculate a lot of these predicates was enabled by the probabilistic opponent tracking module, which allowed the agent to reason about the most likely position of an opponent, even when they were out of visible range. A full list of these predicates is shown in Table 2.

Goal Determination The baseline agent's goal determination is a simple, two-state check: if the team is winning by more than 10 points, defend; otherwise, attack. This is inflexible and fails to adapt to the dynamic, cooperative nature of the game. My implementation replaces this with a multi-tiered, role-based heuristic system, which is split based on the agent's current state as either a Pacman (offense) or a Ghost (defense).

For a Ghost, the logic is a dynamic role-assignment system for defense and offense. The primary goal is to defend the home territory using a *goalChase*. This is not a simple "chase anything" logic; it is highly cooperative, using the PDDL predicates `(ally_chasing, ?e)` and `(closer_to_enemy, ?e)` to ensure only one agent—the one best positioned—chases an invader. Invaders are also prioritized based on their "fatness" (e.g., `(fat_agent_gt10, ?e)`). If the teammate is already handling all defensive duties, this agent is "freed" to go on offense (*goalEatFood*). This entire system is modulated by the game state: a significant lead `(winning_gt3)` biases the agent towards defense, while a significant deficit `(losing_gt3)` biases it towards offense.

For a Pacman, the logic functions as a state machine based on "fatness" (food carried). The default goal is *goalEatFood* to gather points. This is opportunistically upgraded to *goalEatCapsule* if the agent is the closest to a capsule. Once the agent is "fat" (i.e., the `(fat_agent, ?a)` predicate becomes true), the goal flips to *goalEscape* to bank the points. This goal is also triggered early if the agent is "very fat" `(fat_agent_gt10)`, if it is under direct threat `(enemy_close_distance)`, or if it runs out of nearby food.

This dual-system approach, enabled by the richer PDDL state, allows the two agents to dynamically and cooperatively select complementary goals, a significant improvement over the baseline's rigid, identical-agent logic.

PDDL Action Refinement The baseline's PDDL actions are extremely high-level and abstract. The attack action, for instance, has the unrealistic effect of `(not (food_available))`, implying a single action can clear the entire board. This large semantic gap makes it impossible to map the action to a coherent low-level strategy.

My implementation refines this by creating an *action abstraction*, "decomposing" the highly abstract baseline actions (like 'attack') into a more granular, logical set: Instead of one "attack" action, I defined a flow:

- `(:action attack)`: Has the simple effect of `(is_pacman ?a)`. Its job is only to cross the line.
- `(:action eat_food)`: Requires `(is_pacman ?a)` and has the effect of making the agent "fat."
- `(:action escape)`: Requires `(is_pacman ?a)` and has the effect `(not (is_pacman ?a))`.

This refinement is critical. It allows the PDDL solver to generate logical, multi-step plans. For a ghost to achieve a goal of `(fat_agent ?a)`, the solver must first sequence the attack action (to become Pacman) followed by the `eat_food` action. This sequence is far more meaningful for the low-level planner, which now receives a clear, single-purpose command at each step (e.g., "cross the line" or "eat food") instead of the vague "attack" command from the baseline. Similarly, `default_defend` and `chase_enemy` are now two distinct, low-level-mappable actions, allowing the planner to choose between passive patrolling and active pursuit.

2.2.2. Low-Level Planner (Approximate Q-Learning)

The low-level planner retains the approximate Q-learning architecture from the baseline. This is a **model-free reinforcement learning** approach, as it learns the value of state-action pairs (a $Q(s, a)$ function) without building an explicit model of the environment's transition dynamics. The Q-value is estimated as a weighted sum of features:

$$Q(s, a) = \sum_i f_i(s, a) \times w_i$$

The agent selects the action that maximizes this Q-value. However, the baseline's implementation of this architecture is a significant weakness. It uses only three generic sets of weights—*offensiveWeights*, *defensiveWeights*, and *escapeWeights*—to cover all game situations. This forces a single weight set to handle conflicting goals; for example, offensiveWeights must simultaneously value getting closer to food (closest-food) and returning food (chance-return-food). Furthermore, it uses the same *defensiveWeights* for both passive patrol and active defence, which are strategically distinct.

My implementation resolves this conflict by creating a one-to-one mapping between the six refined PDDL actions and six specialized sets of Q-learning weights. This is the most critical low-level improvement. Each weight set is initialized with hand-tuned values to establish strong feature priorities, which are then refined during training via the Q-learning process. This specialization ensures each action starts with a strong bias towards its specific task.

Common features for all actions include not dying and maintaining distance to threatening enemies. However, each action considers its own unique set of features relevant to its specific task. For example, a feature for `chase_enemy` is `distance-to-enemy` whereas a feature for `eat_food` is `distance-to-nearest-food`. A full list of features for each action is shown in Appendix A.0.2.

Task-Specific Rewards and Weight Updates The specialized weights are refined during training using the standard Q-learning update. The weight w_i for a given feature f_i is updated based on the temporal difference (TD) error:

$$\text{TD error} = \left(R(s) + \gamma \max_{a'} Q(s', a')\right) - Q(s, a)$$

$$w_i \leftarrow w_i + \alpha \cdot \text{TD error} \cdot f_i(s, a)$$

Here, $R(s)$ is the reward from the environment, γ is the discount rate, α is the learning rate, and $\max_{a'} Q(s', a')$ is the agent's estimate of the best possible future value, determined by its current Q-value estimates.

The most significant flaw in the baseline's learning framework is its reward implementation. The baseline provides a basic offensive reward, but rewards for defensive and escape actions are unimplemented. This means the agent never learns how to be a good defender or how to escape.

My implementation corrects this by providing a distinct, non-zero reward function for each of the six PDDL actions. This ensures that the *Q-learning* agent receives a clear, non-conflicting reward signal appropriate for its current task, overcoming the sparse and conflicting rewards of the baseline and allowing it to learn effective policies for all six sub-tasks. For example, moves for the `chase_enemy` actions are rewarded when the agent eats the target enemy, whereas moves for the `default_defend` actions are rewarded when the agent is positioned between the enemy and their home. A full list of rewards for each action is shown in Appendix A.0.2. This task-specific reward structure, allows the agent to effectively learn and optimize the weights for all six of its low-level behaviors, a capability the baseline completely lacks.

2.3. Implementation 2: AdvantageMax

Having met the assignment's requirement of using a high-low-level strategy with PDDL, I developed a second implementation, **AdvantageMax**, that did not use a PDDL solver. This approach was designed to be more fluid and reactive, basing its decisions entirely on a single, powerful heuristic: *Advantage*. This is a *domain-specific heuristic* designed to evaluate the quality of any map position by quantifying territorial control.

At its core, this implementation calculates "advantage" by quantifying which team can reach any important location on the map first. This involves a one-time, per-turn computation that builds a "distance map" for every strategically important tile. The process, detailed in Algorithm 2, is as follows:

1. **Identify Key Tiles:** First, the agent identifies every "key tile" on the map. This includes all allied and enemy food, allied and enemy capsules, all allied and enemy escape points (border tiles), and the current positions of all four agents.
2. **Compute Agent Distances:** For each key tile, the agent computes the maze distance for all four agents (two allies, two opponents) to that tile.
3. **Use Pessimistic Opponent Distance:** This is a crucial step. If an opponent's exact location is unknown, we use the probabilistic belief grid. The "distance" for that opponent is calculated as the *minimum* possible distance from *any* tile in their belief distribution that has a non-zero probability. This creates a pessimistic, worst-case-scenario distance for the opponent.
4. **Factor in Risk:** The distance calculation is modified by the strategic map. If a key tile is located in a "dead-end zone" (as identified by the map preprocessing in Figure 1), the distance from that tile to its articulation point is added to the total distance. This risk penalty applies to teammates entering enemy dead-ends and opponents entering allied dead-ends, quantifying the "cost of escape."
5. **Account for Scared Timers:** If an agent is scared, its distance to tiles in the opposing territory is effectively set to its scared timer, simulating that it is not a threat (or is highly vulnerable) for that duration.

The result of this computation is a map, `DistanceMap[tile][agent]`, containing the true "cost" for every agent to reach every important tile. From this, the **advantage** for a specific tile is calculated as:

$$\text{Advantage(tile)} = \min(\text{OpponentCosts}) - \min(\text{AllyCosts})$$

This metric is a form of **differential heuristic**, as it computes the *difference* between the cost for the protagonists (allies) and antagonists (opponents) to reach a goal. A positive value implies a 'safe' tile, while a negative value implies a 'contested' or 'unsafe' one. This single metric serves as the foundation for the entire agent's decision-making, from high-level goal selection (e.g., "which food pellet has the highest advantage?") to low-level movement (e.g., "which move maximizes my advantage?").

The primary drawback of this approach is its computational cost. Calculating this map requires running a "distance-to-all" search for every agent for every single key tile, all on one turn. To mitigate this, the distance map is cached in a class variable, mapping a *gameState* hash to a pre-computed map. This significantly speeds up deliberation in previously-visited states.

2.3.1. Advantage-Based Strategy

In this implementation, we define three high-level "modes" (attack, escape, or defend) based on the computed advantages. The logic is a priority-based ruleset:

1. **Select "escape":** This mode is chosen if the agent is carrying food and either (a) the total food carried by the

Algorithm 2 Distance Map Computation

```
function ComputeDistanceMap(beliefs, topology)
    keyTiles ← all food ∪ all capsules ∪ all escape points ∪ all agent positions
    distMap ← empty map
    for each tile in keyTiles do
        distMap[tile] ← array of size 4
        distToExit ← 0
        if tile is in a dead-end zone then
            exitPoint ← articulation point for tile's dead-end zone
            distToExit ← maze distance from tile to exitPoint
        end if
        for each agent in game do
            scaredTimer ← agent.scaredTimer
            if agent is Teammate then
                dist ← maze distance from agent's position to tile
                if tile is in Enemy Territory then
                    dist ← dist + distToExit
                    dist ← max(scaredTimer, dist)
                end if
            else
                possiblePositions ← non-zero positions from beliefs
                dist ← min(maze distance from pos to tile for pos in possiblePositions)
                if tile is in Home Territory then
                    dist ← dist + distToExit
                    dist ← max(scaredTimer, dist)
                end if
            end if
            distMap[tile][agent] ← dist
        end for
    end for
    return distMap
end function

function GetAdvantage(tile, distMap, allies, opponents)
    minOppCost ← min(distMap[tile][opp] for opp in opponents)
    minAllyCost ← min(distMap[tile][ally] for ally in allies)
    return minOppCost − minAllyCost
end function
```

team is enough to take the lead, or (b) the game timer is low and the agent is at risk of not returning.

2. **Select "attack":** If not escaping, the agent checks if it should attack. This is true only if there is a food pellet with a positive advantage that this agent has a better advantage for than its teammate (advantage > 1 and advantage > teammate's advantage), and the team is not currently winning.
3. **Select "defend":** If neither of the above conditions is met, the agent defaults to defense.

This high-level mode is then passed to the low-level planner, which is a greedy, one-step lookahead search. This is a form of **suboptimal, rule-based search** that prioritizes immediate, high-value moves over a guaranteed optimal path, a necessary trade-off in a real-time, *dynamic environment* with moving obstacles. It evaluates each legal action by simulating the successor state and calculating its heuristic value. The evaluation heuristic is specific to the active mode:

- **For "escape":** The planner's goal is to find a "safe" path home. It evaluates each successor state by checking the advantage of all escape points. An action is considered viable if it leads to a state where at least one escape point has an advantage score greater than 1 (meaning it is a "safe" path). If multiple actions lead to safe paths, it selects the one that gets it closest to its nearest safe escape point. If all actions lead to "trapped" states, it attempts to find a move that is not an immediate death, prioritizing any move that doesn't put it 1 step away from a ghost.
- **For "attack":** The planner's goal is to maximize offensive advantage. It first checks if any move leads to an "eating" state (i.e., the agent's new position is on a food pellet). If so, it prioritizes the move that leads to a state with the most subsequent high-advantage food pellets. If no move results in "eating," it enters a "searching" state and picks the move that minimizes the maze distance to the nearest available food.
- **For "defend":** The planner's goal is to maximize territorial control. It evaluates each successor state by counting the number of allied food and capsule pellets for which the team has a positive advantage. The action that leads to the state with the highest "controlled tile" count is chosen.

This strategy also employs a sophisticated tie-breaking system. If several low-level moves result in the same heuristic value (e.g., two moves both control the same number of food pellets), the agent uses secondary heuristics. It will prefer the move that:

1. Maximizes the magnitude of the advantage, not just the count.
2. Maximizes the distance to its teammate, to spread out and cover more ground.
3. Minimizes the distance to the most "uncertain" opponent (i.e., the one with the widest belief distribution), thereby gaining more information.

2.4. Evaluation and Comparison

To evaluate the two main approaches, both implementations were benchmarked against the baseline `staffTeam.py` and against each other. The tournament leaderboard rank is also included as a measure of performance against a wide variety of unseen opponents. The results are summarized in Table 1.

The results clearly indicate that **AdvantageMax** is the stronger overall competitor. This comparison also provides insight into different planning paradigms. The **CoopPDDL** agent, with its *model-free* Q-learning low level, required extensive training to learn good policies. In contrast, the **AdvantageMax** agent used a **model-based approach**; by using its knowledge of the maze and agent positions (the

Table 1. Performance Comparison of Implementations

Metric	CoopPDDL	AdvantageMax
Win % vs. Staff	92% (46-0-4)	80% (40-1-9)
Avg. Move Time	∼0.0007 s	∼0.0095 s
Peak Rank (ELO)	8th	1st

model) to compute the Advantage heuristic, it could 'look ahead' and make strong decisions with zero training.

Despite a slightly lower direct win rate against the baseline, its peak leaderboard rank of 1st (compared to 8th for **CoopPDDL**) demonstrates a much higher capability against a wider range of skilled opponents. This suggests the *model-based* heuristic approach was far more robust and effective in this complex, multi-agent domain.

This strength is confirmed in a direct head-to-head match (50 games), where **AdvantageMax** (Red) dominated **CoopPDDL** (Blue). **AdvantageMax** achieved a 48% win rate (24 wins) with an average score of +3.22, while **CoopPDDL** only managed a 2% win rate (1 win). The remaining 25 games (50%) were ties, highlighting the robust defensive capabilities of both agents.

The table also reveals interesting strategic differences. **AdvantageMax**'s 80% win rate against the baseline (compared to **CoopPDDL**'s 92%) is not a sign of weakness, but a different playstyle. It played far more defensively, securing 9 ties and only 1 loss, whereas **CoopPDDL** was more aggressive. This suggests the Advantage heuristic creates a more robust, "do-not-lose" strategy.

The most significant trade-off is performance. **AdvantageMax**'s deliberation time is an order of magnitude higher (∼0.0095s vs. ∼0.0007s). This is an expected cost from the expensive, per-turn computation of the distance map, which, even with caching, must be partially recomputed whenever agents enter new, unseen game states. Ultimately, the superior leaderboard performance and head-to-head dominance justify this computational cost, validating the Advantage heuristic as the more powerful and robust approach.

3. Discussion and Reflection

The evaluation in Section 2.4 confirms that the **AdvantageMax** agent is the superior implementation, demonstrating robust, high-level performance against a wide array of opponents. This section provides a deeper analysis of this outcome, reflects on the lessons learned from the two successful agents, and critically examines several alternative—and ultimately unsuccessful—deep reinforcement learning approaches that consumed a significant portion of the project's development time.

3.1. Heuristics vs. Planning: Why AdvantageMax Succeeded

The relative performance of **CoopPDDL** and **AdvantageMax** provides a clear insight: for this specific problem, a single, powerful, *domain-specific heuristic* was more effective than a formal, two-stage planning architecture.

The **CoopPDDL** agent, while fulfilling the assignment's PDDL requirements, was fundamentally a *decoupled planning* system. This architecture created a "semantic gap" between the high-level PDDL action (e.g., `eat_food`) and the low-level planner. This planner then had to decide how to achieve that goal, and while our implementation used the suggested approximate Q-learning with specialized weights, it was still reliant on learning a complex policy to interpret a vague command.

Conversely, the **AdvantageMax** agent was a more *model-based* system. By calculating the 'Advantage' heuristic, it effectively used its model of the world (maze layout, agent positions) to perform a greedy, one-step lookahead search. This heuristic (a form of **differential heuristic** calculating the difference in path cost for ally vs. opponent) was a perfect proxy for territorial control. It collapsed the complex, multi-step problem of "should I attack?" into a single, computable query: "is there any food I can reach safely (i.e., with positive advantage)?" This approach was computationally more expensive per-step but strategically far more sound, justifying its 1st-place rank.

3.2. Alternative Deep RL Approaches and Their Failures

A significant majority of the project time was spent attempting to create a single, end-to-end deep reinforcement learning agent that could learn a winning policy from scratch, removing the need for hand-crafted heuristics. These attempts were inspired by landmark research from DeepMind [2, 3]. This effort ultimately failed due to a combination of three key challenges.

3.2.1. Challenge 1: State Representation

The first challenge was encoding the complex, variable-sized, and partially-observable game state into a fixed-size tensor for a neural network. Two models were developed for this purpose:

1. **Graph Neural Network (GNN):** A GNN (using 'SAGEConv' layers [4]) was implemented. This model leveraged the topological map abstraction, treating the maze as a graph. For each state, a node feature matrix was constructed. Each node's feature vector (11-dimensional) encoded static cell properties (7 one-hot features for wall, food, capsule, etc.) and dynamic properties (4 features for the presence of the agent, the teammate, and the belief probabilities for each of the two opponents).

2. **Convolutional Neural Network (CNN):** A CNN with an actor-critic architecture was also built. This treated the map as an "image" with multiple channels for spatial features (walls, food, ally positions, enemy beliefs) and concatenated the flattened output with a separate vector of scalar features (scared timers, food carried) before feeding it into policy and value heads.

While functional, both models were extremely parameter-heavy. The GNN, for example, had to run message-passing over hundreds of nodes at every time step. This complexity meant that any learning algorithm using them would be data-hungry and slow to converge, especially when starting from a random initialisation.

3.2.2. Challenge 2: Sparse Rewards (The AlphaZero Problem)

One of the learning algorithms attempted was an AlphaZero-style Monte Carlo Tree Search (MCTS) [2]. This approach is ideal for perfect-information games like Go, where every game has a clear win/loss/draw outcome that can be back-propagated.

This approach failed due to a fundamental mismatch in the reward structure. In Pacman CTF, the only game-winning "reward" is scoring, which is a complex, multi-step sequence: cross the midline, navigate to food, eat it, navigate back, and cross the midline again, all while avoiding ghosts. The probability of a randomly-initialised agent ever completing this sequence is near zero. As a result, the MCTS search tree had no positive signals to propagate, and the agent had no way to learn that "moving toward food" was a good action. This sparse reward problem made the MCTS-based approach unusable without a dense, hand-crafted "tie-breaking" heuristic, which would defeat the purpose of learning from scratch.

3.2.3. Challenge 3: Credit Assignment (The PBT Attempt)

To combat the sparse reward problem, a Population-Based Training (PBT) framework was implemented, inspired by [3]. This approach attempted to solve the "credit assignment" problem not by learning the policy directly, but by *evolving the reward function itself*.

This system worked at a meta-level: a population of GNN-based agents, each with a different configuration of reward weights (e.g., `ate_food`, `distance_to_food`) and hyperparameters (e.g., `learning_rate`), competed against each other. Their ELO was tracked, and an "exploit and explore" mechanism periodically replaced the worst-performing agents with mutated copies of the best-performing agents.

While this did allow the agent to learn the value of dense, "shaping" rewards (like `distance_to_food`), the system as a whole failed to converge. The underlying issue remained: even when evolving the reward function, the foundational "scoring" signal was so rare that the ELO-based fitness metric was mostly noise. The system spent all its time learning how to "lose less" or "survive longer" rather than how to win. This, combined with the computationally expensive GNN representation, made the training process intractably slow.

3.3. Key Reflections and Future Work

The development process, including these significant failures, yielded several key lessons.

1. **Heuristics are not a compromise; they are a tool.** The most complex, state-of-the-art algorithms (Deep RL, MCTS) failed because they could not overcome the "blank slate" problem in a sparse-reward, real-time domain. The simple, well-designed heuristic in **AdvantageMax** was the key to victory.
2. **State abstraction is critical.** The 'Strategic Map Preprocessing' was arguably the most valuable component developed, as it was used by all successful and semi-successful agents. It allowed planners to reason about "chokepoints" and "corridors" instead of a raw grid, a form of *abstraction* that the deep-learning models failed to discover on their own.
3. **The lack of temporal memory is a flaw.** A final reflection is that all attempted models (including the final CNN and GNN) processed only a single snapshot of the game state. This lack of a temporal "memory" (e.g., via an RNN/LSTM) meant agents could be baited into loops or fail to reason about an opponent's long-term trajectory.

If this project were to be continued, the most promising path would be a hybrid one: using the **AdvantageMax** agent as a "teacher" to provide a strong, bootstrapped starting policy for a deep reinforcement learning agent, which could then refine its policy via self-play to discover non-obvious, long-term strategies.

4. Conclusion

This report detailed the design, implementation, and evaluation of two distinct AI controllers for the Pacman Capture the Flag challenge. The primary goal was to develop a winning agent, a task that culminated in the **AdvantageMax** implementation achieving a first-place rank on the ELO leaderboard.

The journey to this result involved a direct comparison between two major architectures. The **CoopPDDL** agent, which successfully met the assignment's PDDL requirements by integrating a cooperative, high-level planner with a low-level Q-learning system, proved to be a capable mid-level competitor. However, its rigid, decoupled planning architecture was ultimately outperformed by the heuristic-driven **AdvantageMax** agent.

This outcome provides the report's central conclusion: a well-engineered, *domain-specific heuristic* (the "Advantage" metric) proved to be a more powerful, flexible, and robust strategy than a formal, multi-stage planning system. The **AdvantageMax** agent's ability to collapse complex strategic decisions—such as food safety and territorial control—into a single, computable value allowed it to react more fluidly and effectively to the dynamic, multi-agent environment.

This lesson was further reinforced by the documented failures of several complex, end-to-end deep reinforcement learning approaches. These attempts, which paired sophisticated GNN and CNN state representations with MCTS and Population-Based Training algorithms, failed to overcome the critical, real-world challenges of sparse rewards and intractable data requirements. This finding strongly suggests that for a complex domain like Pacman CTF, intelligent *state abstraction* (like the topological map) and clever *heuristic design* were far more critical to success than the raw power of a learning algorithm.

Ultimately, this project demonstrates that while PDDL provides a formal basis for planning and Deep RL offers a powerful learning paradigm, the most effective solution was a model-based heuristic approach. The most promising path forward would be to combine these strengths, using the **AdvantageMax** agent's powerful heuristic to bootstrap a more advanced reinforcement learning agent, effectively guiding it through the sparse-reward landscape that it could not navigate alone.

AI Acknowledgement

In completing this assignment I used generative AI tools in limited, supporting roles. Specifically, AI was used to:

- Assist with implementation by translating pseudocode and algorithm descriptions into working code.
- Provide help with debugging during development.
- Reword and clean sections of this report for clarity, grammar, and readability.

All design decisions, experimental setups, theoretical analyses, and the overall structure and content of the report were my own. The use of AI was supplementary and did not replace my responsibility for understanding, implementing, and critically analysing the methods presented.

References

[1] J. DeNero and D. Klein, "The pacman ai projects," http://inst.eecs.berkeley.edu/~cs188/sp09/pacman.html, University of California, Berkeley, 2009, accessed: November 5, 2025. 1

[2] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, Oct. 2017. 7, 8

[3] M. Jaderberg, W. M. Czarnecki, I. Dunning, L. Marris, G. Lever, A. G. Castañeda, C. Beattie, N. C. Rabinowitz, A. S. Morcos, A. Ruderman, N. Sonnerat, T. Green, L. Deason, J. Z. Leibo, D. Silver, D. Hassabis, K. Kavukcuoglu, and T. Graepel, "Human-level performance in 3d multiplayer games with population-based reinforcement learning," *Science*, vol. 364, no. 6443, p. 859–865, May 2019. [Online]. Available: http://dx.doi.org/10.1126/science.aau6249 7, 8

[4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf 7

A. Details for Implementation 1

A.0.1. PDDL Predicate Definitions

Table 2 shows the predicates used in this implementation.

Table 2. PDDL Predicate Definitions for Implementation 1

Predicate	Description
Game State & Time Predicates	
`(winning)`	The agent's team has a positive score.
`(winning_gt3)`	The agent's team score is greater than `small_lead_threshold`.
`(losing_gt3)`	The agent's team score is less than -`small_lead_threshold`.
`(low_time_remaining)`	The game has less than 300 time steps remaining.
`(very_low_time_remaining)`	The game has less than 100 time steps remaining.
Food & Capsule Predicates	
`(closest_to_capsule)`	Agent is closer to a power capsule than any estimated opponent.
`(near_food, myObj)`	Agent is within `CLOSE_DISTANCE` of a food pellet.
`(food_available)`	There is at least one food pellet remaining on the enemy side.
General Agent Status Predicates (Apply to all agents)	
`(is_scared, agent_obj)`	The specified agent's `scaredTimer` is greater than 0.
`(is_pacman, agent_obj)`	The specified agent is currently in their enemy's territory.
`(fat_agent, agent_obj)`	Agent's `numCarrying` (adjusted by team score) is > 0.
`(fat_agent_gt3, agent_obj)`	Agent's `numCarrying` (adjusted by team score) is > `small_lead_threshold`.
`(fat_agent_gt10, agent_obj)`	Agent's `numCarrying` (adjusted by team score) is > `large_lead_threshold`.
Enemy-Specific Predicates (Based on Belief Tracking)	
`(enemy_breathing_distance)`	An enemy (not "low" scared) is within `BREATHING_DISTANCE` of the agent.
`(enemy_close_distance)`	An enemy (not "low" scared) is within `CLOSE_DISTANCE` of the agent.
`(enemy_medium_distance)`	An enemy (not "low" scared) is within `MEDIUM_DISTANCE` of the agent.
`(enemy_carrying_food, enemy_obj)`	The specified enemy agent is carrying > 0 food.
`(enemy_past, enemy_obj)`	The specified enemy's estimated position is on the agent's home side.
Teammate Coordination & Relative Position Predicates	
`(ally_chasing, enemy_obj)`	Teammate's current high-level action is `chase_enemy` against this enemy.
`(ally_defending)`	Teammate's current high-level action is `default_defend`.
`(further_back)`	Agent is "further back" (closer to home territory) than its teammate.
`(closer_to_enemy, enemy_obj)`	Agent is closer to this enemy's estimated position than its teammate is.
`(closer_to_closest_enemy)`	Agent's distance to its closest enemy is $\leq$ its teammate's distance to their closest enemy.

A.0.2. Low-Level Features and Rewards

This section details the features and corresponding reward signals used to train the approximate Q-learning agent for each of its six low-level PDDL actions.

Action: `attack` This action's goal is to cross from the home territory into enemy territory.

- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten by an enemy.
  - `breathing-distance-ghosts`: (Count) Number of non-scared enemies 1 step away.
  - `in-enemy-territory`: (Binary) 1 if the agent is in enemy territory.
  - `close-distance-ghosts`: (Count) Number of non-scared enemies within `CLOSE_DISTANCE`.
  - `distance-to-enemy-territory`: (Numeric) Maze distance to the nearest border position.

  - `distance-to-teammate`: (Numeric) Maze distance to the teammate.
  - `stop-reverse`: (Binary) 1 if the action is 'Stop' or reverses direction.
- **Rewards:**
  - Agent enters enemy territory: +10
  - Agent is eaten: -1000
  - Enemy 1 step away: -50 (per ghost)
  - Enemy within close distance: -5 (per ghost)

Action: `eat_food` This action's goal is to consume food pellets while in enemy territory.
- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten.
  - `breathing-distance-ghosts`: (Count) Number of non-scared enemies 1 step away.
  - `ate-food`: (Binary) 1 if the agent ate food in this step.
  - `distance-to-nearest-food`: (Numeric) Maze distance to the nearest food pellet.
  - `close-distance-ghosts`: (Count) Number of non-scared enemies within `CLOSE_DISTANCE`.
  - `distance-to-attacking-teammate`: (Numeric) Maze distance to teammate (if also `eat_food`).
  - `stop-reverse`: (Binary) 1 if the action is 'Stop' or reverses direction.
- **Rewards:**
  - Agent eats a food pellet: +100
  - Agent is eaten: -1000
  - Enemy 1 step away: -100 (per ghost)
  - Enemy within close distance: -10 (per ghost)
  - Time step penalty (for efficiency): -1

Action: `eat_capsule` This action's goal is to consume a power capsule.
- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten.
  - `breathing-distance-ghosts`: (Count) Number of non-scared enemies 1 step away.
  - `ate-capsule`: (Binary) 1 if the agent ate a capsule in this step.
  - `distance-to-nearest-capsule`: (Numeric) Maze distance to the nearest capsule.
  - `close-distance-ghosts`: (Count) Number of non-scared enemies within `CLOSE_DISTANCE`.
  - `stop-reverse`: (Binary) 1 if the action is 'Stop' or reverses direction.
- **Rewards:**
  - Agent eats a power capsule: +500
  - Agent is eaten: -1000
  - Enemy 1 step away: -100 (per ghost)
  - Time step penalty (for efficiency): -1

Action: `escape` This action's goal is to return to the home territory to score carried food.
- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten.
  - `in-home`: (Binary) 1 if the agent is in its home territory.
  - `breathing-distance-ghosts`: (Count) Number of non-scared enemies 1 step away.
  - `distance-to-home`: (Numeric) Maze distance to the nearest home territory position.
  - `close-distance-ghosts`: (Count) Number of non-scared enemies within `CLOSE_DISTANCE`.
  - `stop-reverse`: (Binary) 1 if the action is 'Stop' or reverses direction.
- **Rewards:**
  - Agent successfully returns home: +200
  - Additional reward per food pellet returned: +100
  - Agent is eaten (high penalty): -2000
  - Enemy 1 step away: -200 (per ghost)
  - Enemy within close distance: -20 (per ghost)
  - Time step penalty (for efficiency): -1

Action: `chase_enemy` This action's goal is to actively pursue and eat a specific enemy Pacman.

- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten.
  - `ate_enemy`: (Binary) 1 if the agent ate the target enemy.
  - `distance-to-enemy`: (Numeric) Maze distance to the target enemy.
  - `between-enemy-and-escape`: (Binary) 1 if agent is on the path between the enemy and their home.
  - `distance-to-teammate`: (Numeric) Maze distance to the teammate.
  - `stop-reverse`: (Binary) 1 if the action is 'Stop' or reverses direction.
- **Rewards:**
  - Agent eats the target enemy: +500
  - Agent is positioned between enemy and their escape: +10
  - Agent is eaten: -1000
  - Agent leaves home territory: -50
  - Time step penalty (for efficiency): -1

Action: `default_defend` This action's goal is to patrol the home territory and intercept invaders.

- **Features:**
  - `got-eaten`: (Binary) 1 if the agent was eaten.
  - `in-home`: (Binary) 1 if the agent is in its home territory.
  - `distance-to-nearest-enemy`: (Numeric) Maze distance to the nearest enemy Pacman.
  - `distance-to-teammate`: (Numeric) Maze distance to the teammate.
  - `distance-to-teammate-both-defending`: (Numeric) Distance, but only active if both are defending.
- **Rewards:**
  - Agent eats an invading enemy: +300
  - Agent remains in home territory: +10
  - Agent is eaten: -500
  - Agent leaves home territory: -50
  - Time step penalty (for efficiency): -1